SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I B vijay karthikkayan (192511348) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python's scikit-learn library.

(a) Load, Pre-process, and Split the Dataset

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(df.head())
print(df.dtypes)
print(df.isnull().sum())        # check for missing values

df['species_encoded'] = iris.target         # label-encode target
X = df[iris.feature_names]
y = df['species_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test  size=0.2, random  state=42, stratify=y)
```

Output — first 5 rows:

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
0                5.1               3.5                1.4               0.2  setosa
1                4.9               3.0                1.4               0.2  setosa
2                4.7               3.2                1.3               0.2  setosa
3                4.6               3.1                1.5               0.2  setosa
4                5.0               3.6                1.4               0.2  setosa
```

Output — data types:

```
sepal length (cm)     float64
sepal width (cm)      float64
petal length (cm)     float64
petal width (cm)      float64
species              category
```

The dataset contains no missing values, so imputation is not required. The only categorical field is the target column 'species', which is label-encoded (0 = setosa, 1 = versicolor, 2 = virginica) for use with scikit-learn. The four numeric features (sepal length, sepal width, petal length, petal width) require no further encoding.

The dataset (150 samples) is split 80/20 with stratification, giving 120 training samples and 30 testing samples, preserving the 1:1:1 class ratio in both sets.

(b) Build the Decision Tree (Gini Index)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)

print(export_text(clf, feature_names=list(X.columns)))
```

Output — tree structure:

```
|--- petal length (cm) <= 2.45
|   |--- class: 0  (setosa)
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1  (versicolor)
```

```
|   |--- petal length (cm) >  4.95
|   |   |--- sepal length (cm) <= 6.15
|   |   |   |--- sepal width (cm) <= 2.45  --> class: 2 (virginica)
|   |   |   |--- sepal width (cm) >  2.45  --> class: 1 (versicolor)
|   |   |--- sepal length (cm) >  6.15  --> class: 2 (virginica)
|   |--- petal width (cm) >  1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.00  --> class: 2 (virginica)
|   |   |   |--- sepal width (cm) >  3.00  --> class: 1 (versicolor)
|   |   |--- petal length (cm) >  4.85  --> class: 2 (virginica)
```

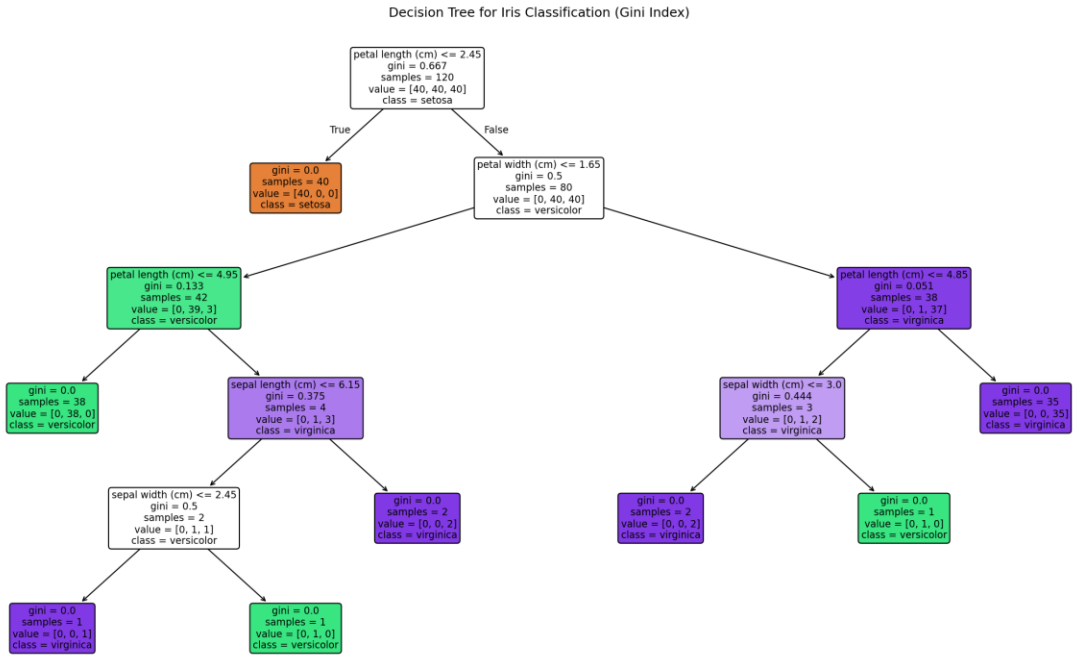

Figure 1: Visualised Decision Tree (Gini criterion) for the Iris dataset

Root node identification & justification

The root node splits on petal length (cm) ≤ 2.45, with a Gini impurity of 0.667 at the root.

Feature importance ranking computed by the trained tree:

- petal length: 0.559 (highest — chosen as the root split)
- petal width: 0.406
- sepal width: 0.029
- sepal length: 0.006

Petal length is selected as the root because it produces the greatest reduction in Gini impurity among all candidate splits at that node: a threshold of 2.45 cm perfectly separates the 40 setosa samples (Gini = 0) from the remaining 80 versicolor/virginica samples in a single split. This matches the well-known biological fact that Iris-setosa has markedly smaller petals than the other two species, making petal length the most discriminative feature and therefore the greedy CART algorithm's first choice.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Output:

```
Accuracy: 0.9333
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```

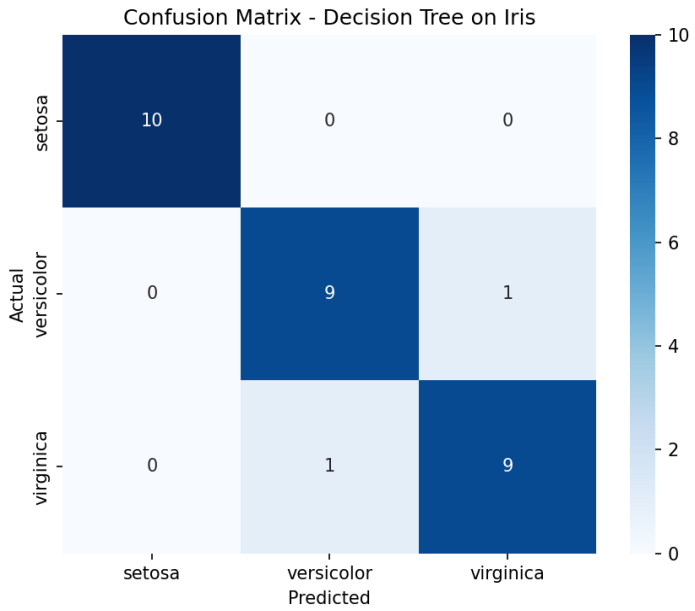

Figure 2: Confusion matrix heat-map for the test set

Performance comment

The model achieves 93.3% accuracy on the held-out test set. Setosa is classified perfectly (precision = recall = 1.00) because it is linearly separable from the other two species on petal length alone. All errors occur between versicolor and virginica, which is expected because these two species overlap in petal/sepal measurements near the decision boundary; each records one false positive and one false negative, giving both an identical precision/recall/F1 of 0.90.

Misclassified instances (at least two, as required):

- Test index 134 — features [sepal length 6.1, sepal width 2.6, petal length 5.6, petal width 1.4]: true label = virginica, predicted = versicolor. The unusually short petal length (5.6) and narrow sepal width for a virginica sample push it across the learned boundary into the versicolor region.
- Test index 77 — features [sepal length 6.7, sepal width 3.0, petal length 5.0, petal width 1.7]: true label = versicolor, predicted = virginica. This sample's petal width (1.7) exceeds the tree's 1.65 threshold used to separate versicolor from virginica, so it is routed into the virginica branch despite being labelled versicolor.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4×4 Grid World and observe the agent's learned policy.

(a) Environment Setup

A 4×4 Grid World is defined with 16 states numbered 0–15 (row-major order), 4 actions (UP, DOWN, LEFT, RIGHT), a start state, a goal state, and one obstacle/penalty state:

```
 0  1  2  3
 4  5  6  7
 8  9 10 11
12 13 14 15

Start state  = 0   (top-left)
Goal state   = 15  (bottom-right, terminal, reward = +10)
Obstacle     = 5   (penalty cell, reward = -5)
Every other move: reward = -1  (step cost, encourages shortest path)

Hyperparameters: alpha (learning rate) = 0.1, gamma (discount) = 0.9,
epsilon (exploration, epsilon-greedy) = 0.2, episodes = 100

import numpy as np

N_STATES, N_ACTIONS, GRID_SIZE = 16, 4, 4
START_STATE, GOAL_STATE, OBSTACLE_STATE = 0, 15, 5
action_names = {0:'UP', 1:'DOWN', 2:'LEFT', 3:'RIGHT'}

def next_state(s, a):
    row, col = divmod(s, GRID_SIZE)
    if a == 0:   row = max(row-1, 0)
    elif a == 1: row = min(row+1, GRID_SIZE-1)
    elif a == 2: col = max(col-1, 0)
    elif a == 3: col = min(col+1, GRID_SIZE-1)
    return row*GRID_SIZE + col

def get_reward(s):
    if s == GOAL_STATE: return 10
    if s == OBSTACLE_STATE: return -5
    return -1

Q = np.zeros((N_STATES, N_ACTIONS))
alpha, gamma, epsilon = 0.1, 0.9, 0.2
```

(b) Running Q-Learning for 100 Episodes

```
for episode in range(1, 101):
    s = START_STATE
    for step in range(100):
        if np.random.rand() < epsilon:
            a = np.random.randint(N_ACTIONS)        # explore
        else:
            a = np.argmax(Q[s])                     # exploit

        s_next = next_state(s, a)
        r = get_reward(s_next)
        Q[s, a] += alpha * (r + gamma*np.max(Q[s_next]) - Q[s, a])  # Bellman update
        s = s_next
        if s == GOAL  STATE:
            break
```

Q-values were recorded at Episodes 1, 50 and 100 for two representative states: State 0 (the start state) and State 10 (a state adjacent to the goal path).

Episode	State	UP	DOWN	LEFT	RIGHT
1	0	-0.297	-0.288	-0.271	-0.200
1	10	-0.100	-0.100	-0.109	-0.100
50	0	-2.277	-2.222	-2.262	-2.216
50	10	-0.235	0.127	-0.109	5.370
100	0	-2.257	-2.281	-2.352	0.191
100	10	-0.235	3.648	0.200	7.920

How the Q-table evolves

- Episode 1: Q-values are close to zero / lightly negative — the agent has only stepped through the environment once via the epsilon-greedy random walk, so almost no reward signal has propagated back through the Bellman update yet.
- Episode 50: Q-values at State 10 have grown sharply for RIGHT (5.370), since State 10 lies close to the goal and the +10 terminal reward has begun propagating backward through repeated visits. State 0 (far from the goal) still shows small negative values, reflecting accumulated step-cost with reward not yet propagated that far back.
- Episode 100: Propagation has reached further — State 0's RIGHT action turns positive (0.191), showing the value signal from the goal has now backed up all the way to the start state. State 10 continues to strengthen (RIGHT = 7.920), approaching its converged estimate of gamma-discounted future reward.

This pattern is the expected behaviour of Q-learning: value information originates at the goal and propagates backward, one Bellman update at a time, so states nearer the goal converge earlier than states farther away.

(c) Final Policy, Reward Curve, and Convergence

```
policy = np.argmax(Q, axis=1)
print(policy.reshape(4,4))        # optimal action index per state
```

Final learned policy (optimal action per state):

```
RIGHT  RIGHT  DOWN   DOWN
DOWN   RIGHT  DOWN   DOWN
RIGHT  DOWN   RIGHT  DOWN
RIGHT  RIGHT  RIGHT  UP    <- GOAL is state 15 (bottom-right)
```

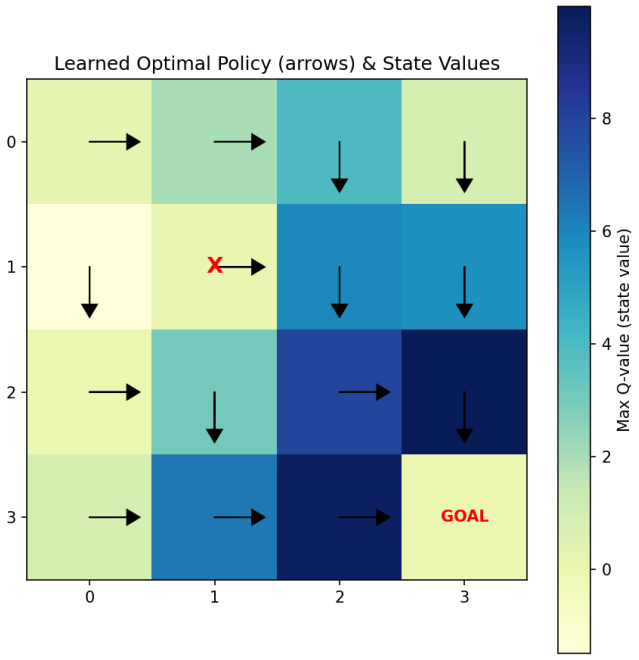

Figure 3: Learned optimal policy (arrows) overlaid on the state-value heat-map. 'X' marks the obstacle (state 5); 'GOAL' marks state 15.

The arrows form a consistent path from every state toward the goal at state 15, routing around the penalty cell at state 5 (row 1, column 1) — e.g. state 1 goes DOWN instead of RIGHT into the obstacle, and state 4 goes DOWN rather than RIGHT into it.

States closer to the goal (bottom-right region) show the highest state values (darkest cells), confirming that value increases monotonically along the optimal path.

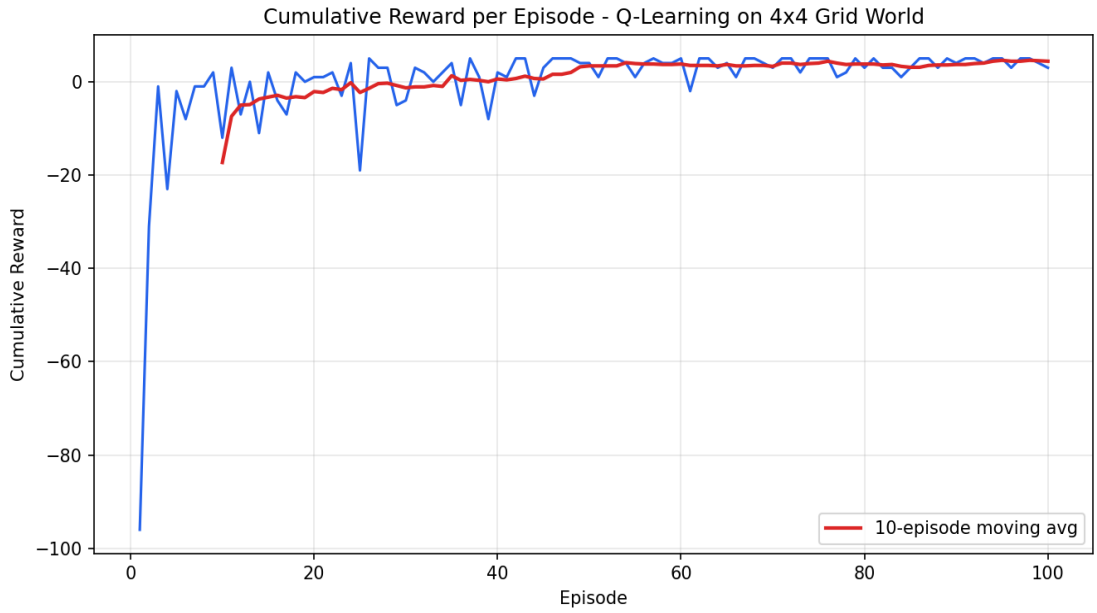

Figure 4: Cumulative reward per episode, with a 10-episode moving average

Convergence behaviour — justification

Episode 1 yields a very large negative cumulative reward (≈ -96) because the fully-random initial policy wanders through the grid for up to 100 steps before (if ever) reaching the goal, accumulating many -1 step penalties. Within the first ~10 episodes the reward rises sharply as the agent starts reaching the goal sooner. From roughly episode 40 onward the moving-average curve flattens out around a small positive value, indicating the greedy component of the policy (argmax Q) has settled on a near-shortest path to the goal. The remaining episode-to-episode fluctuation is due to the epsilon = 0.2 exploration rate, which keeps injecting random actions even after the optimal policy has been learned — this is expected for epsilon-greedy Q-learning and would disappear if epsilon were annealed to 0 or evaluation were run in pure-greedy mode.